Introduction to

Deep Learning

From Neurons to Intelligence

Neural Nets | Math | Hands-on | Roadmap

Today's Roadmap

01 **What is Deep Learning?**
Definition, history & the big picture

02 **The Building Block: Neurons**
Perceptron, activation functions, layers

03 **Neural Network Architecture**
Forward pass, backprop, loss functions

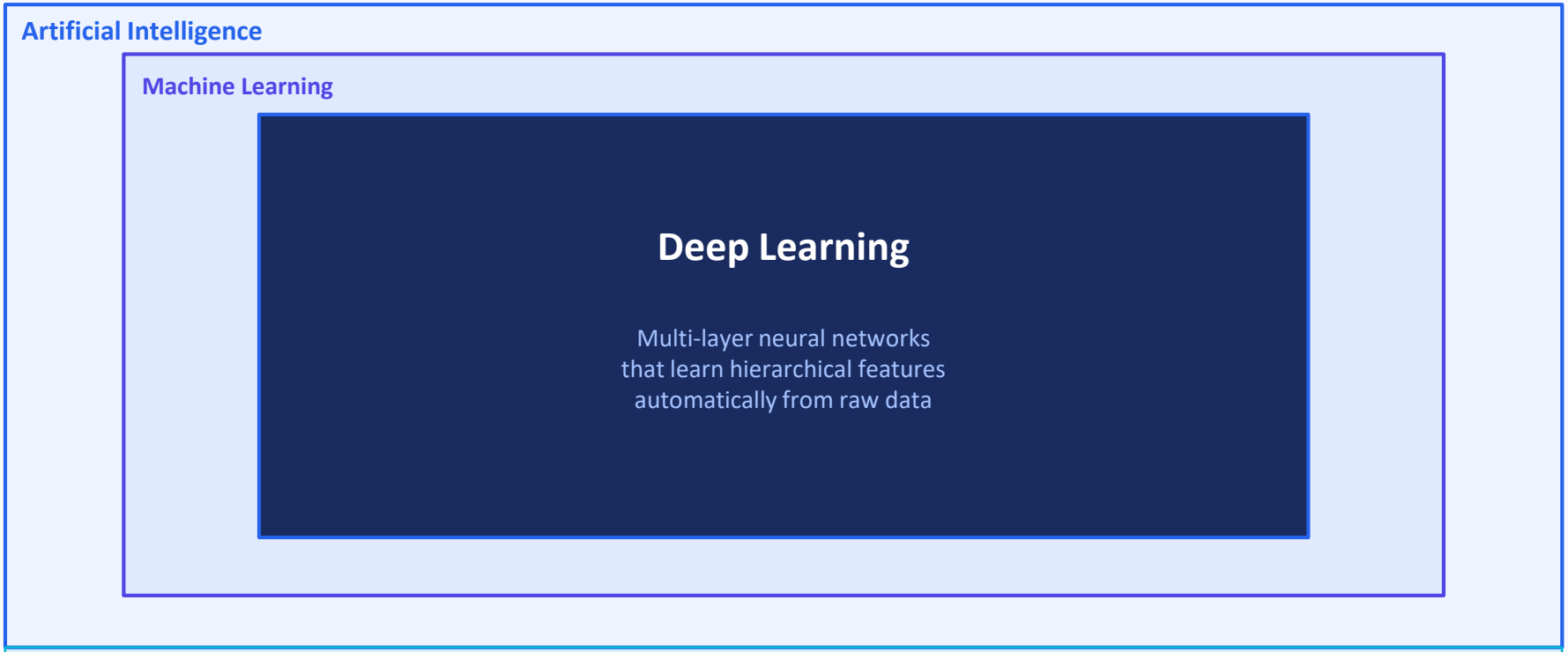
04 **Key Deep Learning Techniques**
Regularization, optimization, dropout

05 **Major Architectures**
CNN, RNN, Transformers, GANs

06 **Real-World Applications**
CV, NLP, healthcare, science

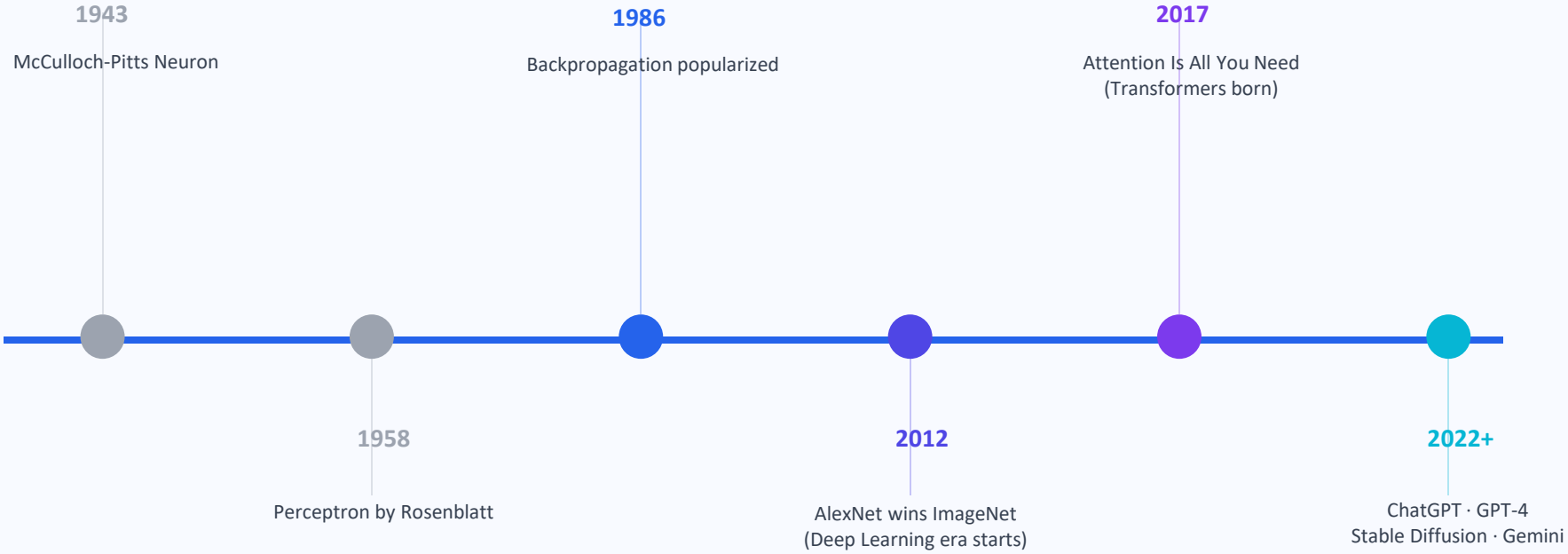
07 **Learning Roadmap**
How to master deep learning step by step

01 / What is Deep Learning?



Deep Learning is a subfield of ML using artificial neural networks with many layers — inspired by the human brain.

01 / Why is Deep Learning Powerful Now?



Big Data

Millions of labeled examples available

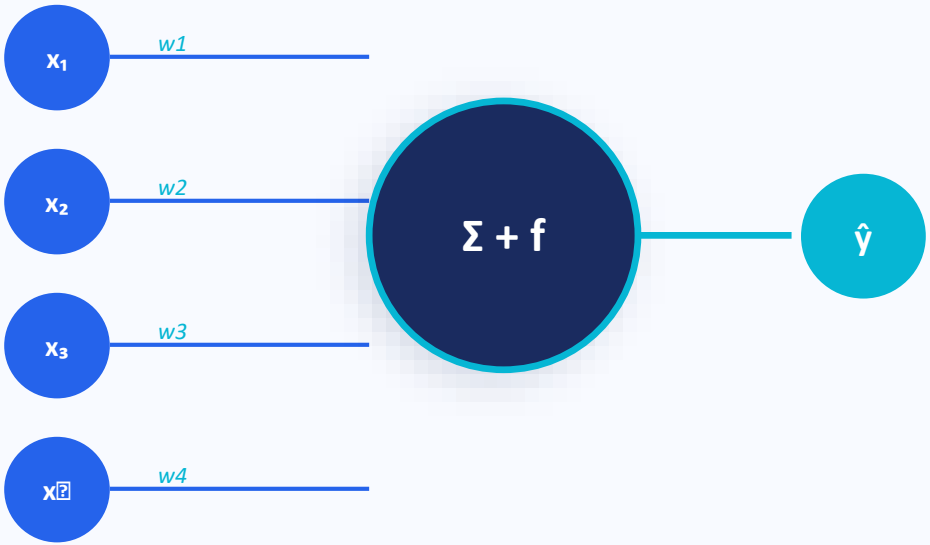
GPU Power

1000x faster training with modern hardware

Better Algorithms

Adam, BatchNorm, Dropout, Residuals

02 / The Building Block: Artificial Neuron

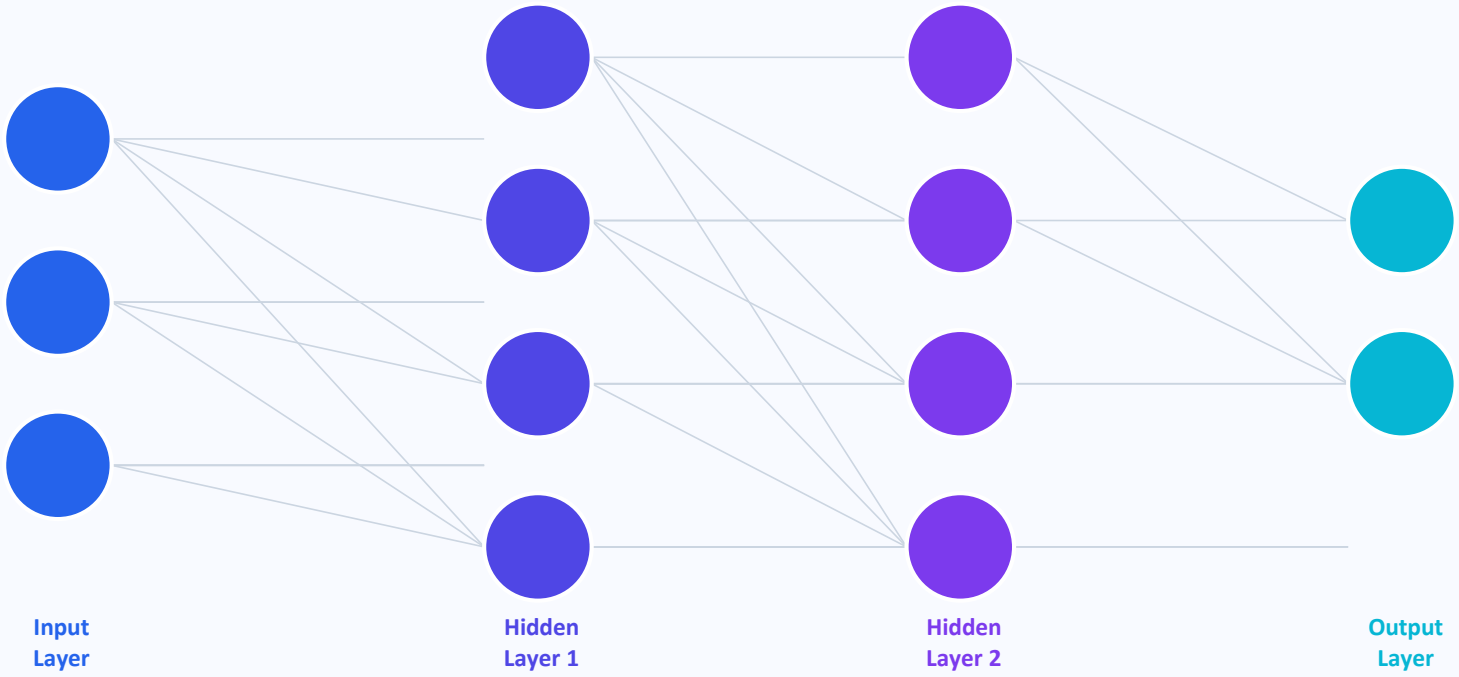


$$\text{output} = f\left(\Sigma\, w_i x_i + b \right)$$

Common Activation Functions

ReLU	$\max(0, x)$	Hidden layers (default)
Sigmoid	$1/(1+e^{-x})$	Binary output
Softmax	$e^{x_i} / \Sigma e^{x_j}$	Multi-class output
Tanh	$\tanh(x)$	Normalized hidden

02 / Neural Network Architecture



Forward Pass

Data flows left → right through the network computing predictions

Backpropagation

Error gradient flows right → left updating weights via chain rule

Loss Function

MSE for regression · Cross-Entropy for classification

Optimizer

SGD · Adam · RMSprop update weights to minimize loss

03 / How Networks Learn: Training Loop

1 Forward Pass

Input → Network → Prediction ŷ

2 Compute Loss

L = Loss(ŷ, y_true)

3 Backpropagation

∂L/∂w for every weight via chain rule

4 Update Weights

w ← w − η · ∂L/∂w

Repeat × Epochs

Loss decreases over epochs as weights improve

04 / Key Deep Learning Techniques

Regularization

- L1 / L2 weight penalties
- Reduces overfitting
- Keeps weights small

Dropout

- Randomly zero out neurons
- Forces redundant learning
- Rate: 0.2 – 0.5 typical

Batch Normalization

- Normalize layer inputs
- Faster training, stable gradients
- Acts as mild regularizer

Optimizers

- SGD: classic, momentum
- Adam: adaptive lr per param
- RMSprop: non-stationary data

Learning Rate

- Too high → diverge
- Too low → slow/stuck
- Schedulers: cosine, step, warmup

Data Augmentation

- Flip, rotate, crop, noise
- Artificially grow dataset
- Reduces overfitting greatly

05 / Major Deep Learning Architectures

CNN

Convolutional Neural Network

Use Case:

Computer Vision

How:

Local filters detect edges → shapes → objects

Examples:

ResNet, VGG, EfficientNet, YOLO

RNN/LSTM

Recurrent / Long Short-Term Memory

Use Case:

Sequences & Time Series

How:

Hidden state carries memory across time steps

Examples:

Text generation, speech, ECG analysis

Transformer

Self-Attention based Architecture

Use Case:

NLP & Beyond

How:

Attention scores relate every token to every other

Examples:

BERT, GPT, ViT, Whisper, CLIP

GAN

Generative Adversarial Network

Use Case:

Generative AI

How:

Generator vs Discriminator adversarial training

Examples:

StyleGAN, CycleGAN, Pix2Pix

06 / Real-World Applications

Healthcare

- Cancer detection in histopathology
- Medical image diagnosis (MRI/CT)
- Drug discovery & genomics

Computer Vision

- Object detection (YOLO, DETR)
- Face recognition & ID
- Autonomous driving

Natural Language

- ChatGPT, Gemini, LLaMA
- Machine translation
- Sentiment & summarization

Audio & Speech

- Speech-to-text (Whisper)
- Music generation (Jukebox)
- Voice cloning

Science

- AlphaFold protein folding
- Climate modeling
- Particle physics (CERN)

Gaming & RL

- AlphaGo / AlphaZero
- Game AI (StarCraft, DOTA)
- Robot locomotion

Math Behind Deep Learning

Linear Algebra

$$z = Wx + b$$

- Weights W are matrices
- z = Wx + b (matrix mul.)
- Efficient on GPU with BLAS

Calculus & Gradients

$$\partial L/\partial w = \partial L/\partial y \cdot \partial y/\partial w$$

- Chain rule: ∂L/∂w = ∂L/∂y · ∂y/∂w
- Gradient = direction of steepest ascent
- We descend: w -= lr · grad

Probability & Stats

$$H(y, \hat{y}) = -\Sigma\, y \log(\hat{y})$$

- Cross-entropy: -Σ y log(ŷ)
- Softmax → probability distribution
- KL divergence for generative models

You don't need to be a math PhD — intuition + practice + libraries (PyTorch/TF) handle the hard parts!

Deep Learning Tools & Ecosystem

Models & APIs	GPT-4	Gemini	Claude	HuggingFace Hub
High-Level Frameworks	Keras	FastAI	Hugging Face Transformers	Lightning
Core Frameworks	PyTorch (most popular)	TensorFlow / Keras	JAX / Flax	
Math / Compute	CUDA (NVIDIA GPU)	NumPy / SciPy	BLAS / cuBLAS	
Hardware	GPU (RTX, A100, H100)	TPU (Google)	Apple Silicon MPS	

07 / Your Deep Learning Roadmap

Phase 1
Foundations

4–6 weeks

- ✓ Python · NumPy · Pandas
- ✓ Linear algebra & calculus refresher
- ✓ ML basics (regression, classification)
- ✓ Scikit-learn projects

Phase 2
Core Deep Learning

6–8 weeks

- ✓ PyTorch fundamentals
- ✓ FNNs from scratch
- ✓ CNNs for image tasks
- ✓ Backprop & optimizers deep dive

Phase 3
Advanced Architectures

8–10 weeks

- ✓ RNNs · LSTMs · GRUs
- ✓ Transformers & Attention
- ✓ Transfer Learning & Fine-tuning
- ✓ GANs & VAEs

Phase 4
Applied & Research

Ongoing

- ✓ Pick a domain (NLP/CV/Med)
- ✓ Kaggle competitions
- ✓ Read & reproduce papers
- ✓ Build & deploy projects

Common Challenges & How to Solve Them

Problem: Overfitting

✓ Dropout · L2 reg · More data · Early stopping

Problem: Vanishing Gradients

✓ ReLU · Batch norm · Residual connections · LSTM

Problem: Slow Training

✓ GPU/TPU · Mixed precision · Larger batch size · Better optimizer

Problem: Not Enough Data

✓ Data augmentation · Transfer learning · Synthetic data · GANs

Problem: Hyperparameter Hell

✓ Grid/random search · Optuna · Learning rate finder · WandB

Problem: Black Box Problem

✓ SHAP · Grad-CAM · Attention maps · LIME explanations

Key Takeaways

- Deep Learning = multi-layer neural networks learning hierarchical features
- Three pillars of the DL revolution: Big Data, GPU Power, Better Algorithms
- Core loop: Forward pass → Loss → Backprop → Update weights (repeat)
- CNNs for images, Transformers for sequences, GANs for generation
- PyTorch is your best friend — learn it deeply
- The roadmap: Foundations → Core DL → Advanced Architectures → Applied Research

"The best time to start learning Deep Learning was yesterday. The second best time is now." — BrainerX AI Masterclass

Transfer Learning & Pre-Trained Models

Large Dataset
(ImageNet, Wikipedia)

Pre-Training
(millions of steps)

Pre-Trained
Model Weights

Your Task
(Fine-Tuning)

Why Transfer Learning?

- ✓ You don't need millions of examples
- ✓ Training from scratch is expensive (days/weeks on GPU)
- ✓ Pre-trained models already know edges, shapes, language patterns
- ✓ Fine-tune in hours instead of training weeks

Fine-Tuning Strategies

Feature Extraction	Freeze all layers, train only the head
Partial Fine-Tune	Unfreeze last N layers + new head
Full Fine-Tuning	Update all weights (needs more data)
LoRA / Adapters	Inject tiny trainable modules (LLM era)

Famous pre-trained models: ResNet-50 · VGG-16 · EfficientNet · BERT · GPT-2 · ViT · CLIP · Whisper

Attention Mechanism & Transformers

The Core Idea

Instead of reading a sentence word-by-word in order, Attention lets the model look at ALL words simultaneously and decide which ones matter most for each prediction.

Scaled Dot-Product Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(QK^T / \sqrt{d_k} \right) \cdot V$$

Q = Query · K = Key · V = Value
d_k = key dimension (scaling prevents gradient vanishing)

Transformer Building Blocks

Multi-Head Attention

Run attention in parallel across multiple subspaces

Add & Norm

Residual connection + Layer normalization

Feed-Forward Network

Two linear layers with ReLU in between

Positional Encoding

Injects word order info (no recurrence needed)

Encoder / Decoder

BERT=Encoder only
GPT=Decoder only

Model Evaluation & Metrics

Classification Metrics

Accuracy	`(TP+TN) / Total`	*Balanced classes*
Precision	`TP / (TP+FP)`	*Cost of false positives high*
Recall	`TP / (TP+FN)`	*Cost of false negatives high*
F1-Score	`2·P·R / (P+R)`	*Imbalanced datasets*
AUC-ROC	`Area under ROC curve`	*Binary classification*

Regression Metrics

MAE	$\text{mean } \lvert y - \hat{y} \rvert$	*Robust to outliers*
MSE	$\text{mean } (y - \hat{y})^2$	*Penalizes large errors*
RMSE	$\sqrt{\text{MSE}}$	*Same unit as target*
R^2	$1 - SS_{res}/SS_{tot}$	*% variance explained*

Underfitting: High train & val loss — model too simple

Good Fit: Low train loss, low val loss — goal!

Overfitting: Low train loss, high val loss — memorizing

Deep Learning in Production 🚀

Data

Collect · Clean · Label · Augment

→

Train

Experiment tracking · Hyperparameter tuning

→

Evaluate

Metrics · Bias checks · Explainability

→

Deploy

REST API · Docker · Cloud (AWS/GCP/Azure)

→

Monitor

Data drift · Model decay · Retraining triggers

Key MLOps & Deployment Tools

Experiment Tracking: MLflow · Weights & Biases · Neptune

Model Serving: FastAPI · TorchServe · TF Serving · Triton

Containerization: Docker · Kubernetes · AWS SageMaker

Monitoring: Evidently · Grafana · Prometheus · Arize

What We'll Learn

— BrainerX AI Masterclass Full Curriculum

01 Python & ML Foundations

NumPy · Pandas · Matplotlib · Scikit-learn · ML theory

02 Neural Networks from Scratch

Perceptron · FNN · Backprop · PyTorch basics · First model

03 Convolutional Neural Networks

Conv layers · Pooling · ResNet · EfficientNet · Image projects

04 Sequence Models & NLP

RNN · LSTM · GRU · Tokenization · Text classification

05 Transformers & LLMs

Attention · BERT · GPT · Fine-tuning · Prompt engineering

06 Generative AI & Advanced

GANs · VAEs · Diffusion models · Stable Diffusion basics

07 Applied Deep Learning

Healthcare AI · Computer Vision projects · Kaggle competitions

08 Deployment & MLOps

FastAPI · Docker · WandB · Cloud deployment · Production tips

Every module includes hands-on projects, code walkthroughs, and real datasets — Let's build together!